File Handling Internals



eh lly zahr lel user ? we eh elly t7t internally mtkhzn ezay ?!

Mmkn file yb2a fragmented `into 3 storage devices`, mmkn file mwgod 3la b3do 3ady w hakza.

Who is responsible for providing that interface?

The **File System. Hoa lly bykhby el details de w ydek 7aga baseta tt3aml m3aha.**

File-System

- Storage Device can be (Hard Disk/Flash/CD), and it is divided into **partitions**.
- Each **partition** is handled by the **File-System**.
- **File-System** divides the Partition into **two sections**

    - File System **Meta-Data** [Meta: Data about Data] (File system overhead)
        - **Tables & Records** for each File
            - [Size, Permissions, Owners, Last modified, date of creation, pointers to data blocks]
    - File System **Data** [**Data Blocks**]
        - One file can be fragmented into multi data blocks maintained by the FS. If there is no contiguous free space.
- What is the relation between them? There is a pointer to each data block.

💡 Analogy

Imagine a library:

Section	Purpose
Metadata	Like the **card catalog** (titles, authors, location of books)
Data	Like the **bookshelves** (actual content)

You need the catalog to find the books — just like a file system needs metadata to find and manage your files.

What is a File ?

FileSystem **Meta-Data**

1- `inode` # (index node) unique ID per the filesystem.

2- File **Size**.

3- **Owner user**, Owner group.

4- **Permissions [** `rwx` **]**

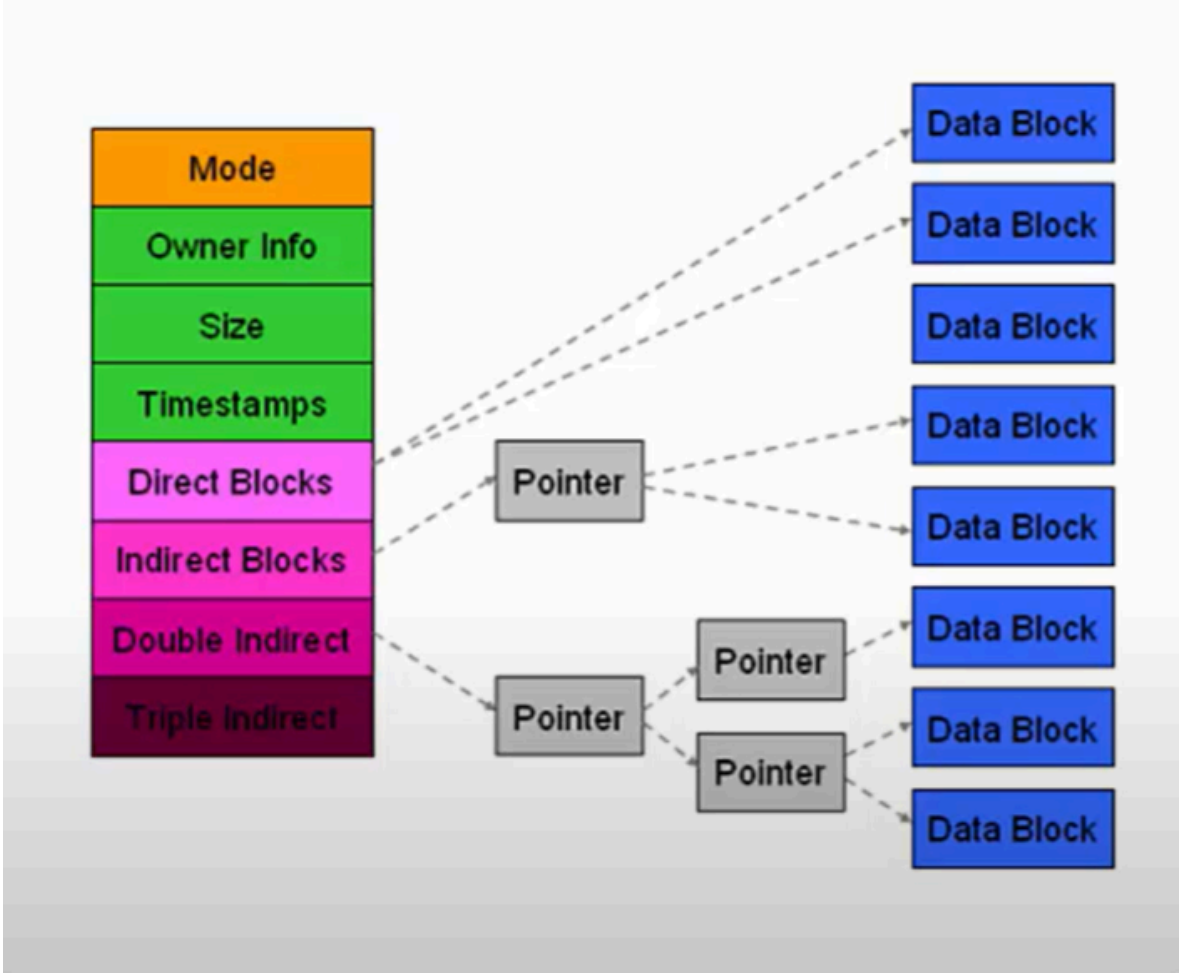
5- Date & time of **creation**.

6- Date & time of the last **modification**.

7- **Data Blocks** (pointer to **data blocks** in the **Data storage**)

Neither Filename nor File path is stored

The “inode” structure



- A file is a **set of bytes** that represent some **content** (pdf document, excel sheet, binary executable, ...)
- The files are stored in (partition) in a storage device as a single data block or fragmented into a group of data blocks (within the same partition).
- The fileSystem is responsible for managing the data block(s), and their representation to the user.
- For this management, the fileSystem needs to maintain some extra info about the file which is called file `Meta-data`
  - File Size
  - File Owner (user & group)

- File permissions
- Data of creation/last modification
- Pointers to the file content data blocks
- etc..

> 💡 These meta-data are stored in an “inode” structure.
> Note: the inode does not contain the file name or its location

- This means that the FS maintains a table of “inode” structures (one structure per file)
- The `“inode”` structure will contain all file meta-data (except its filename & the location)
- The `“inode”` structure will also point to the data blocks of the file.
- Each data node has a **unique number** across the filesystem (the `“inode”` number)
- inode numbers are unique per FS (and not across the system).
- Directories are a special type of files, accordingly, they are treated the same way.

So, What about Filenames & Paths ??

The “**inode**” structures does not have knowledge about the filename.

- Instead, each file or directory has another structure named `“dentry”`, this structure maps a file/directory to its `“inode”`
- kol `dentry` 3ndha `parent` w 3ndha `children`: y3ny el root 3ndha `dentry` wl dentry bta3tha btshawr 3la kol l directories lly t7tha.
- The “dentry” structure forms the directory tree.
- The dentry also helps in caching — speeding up repeated file accesses.

🔄 Relationship Between inode & dentry

- A **dentry maps a file name to an inode**.
- A single **inode can have multiple dentries** pointing to it (via **hard links**).

> 💡 dentry is a misleading name

🧠 Why the Separation Exists ? Hard link

There is no direct way[command] to see the dentry table !

Example

```
/home/aramy/my-file.txt

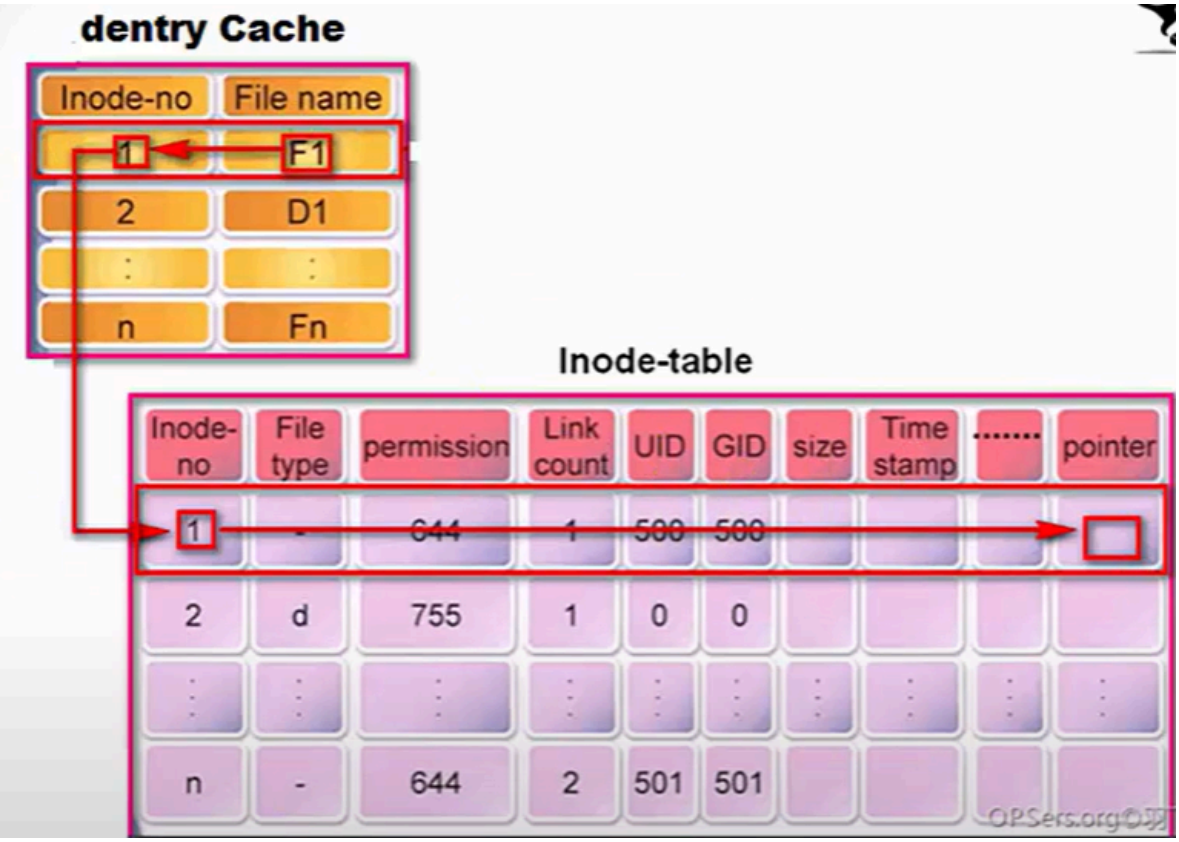
// dentry
Name: /
Parent:
inode#: x

// dentry
Name: home
Parent: /
inode#: y

// dentry
Name: ramy
Parent: home
inode#: z

// dentry
Name: my-file.txt
Parent: ramy
inode#: w
```

```
// inode (w)
File metadata(inode)--> Data Block of the File
```



Linux Commands

```
$ls -i (list with showing the inode number)
$ls -il (list with showing the inode with long foramt)

$stat log (show File status info)
File Name, Size, Blocks, IO block:, Regual File[msh directory
y3ny], Inode, Links
Uid:, Gid, Access, Modify, Change, Birth ....

$df (Show FileSystem Disk Space Usage)
FileSystem           1K-blocks    Used   Available  Use%  Mou
nted on
```

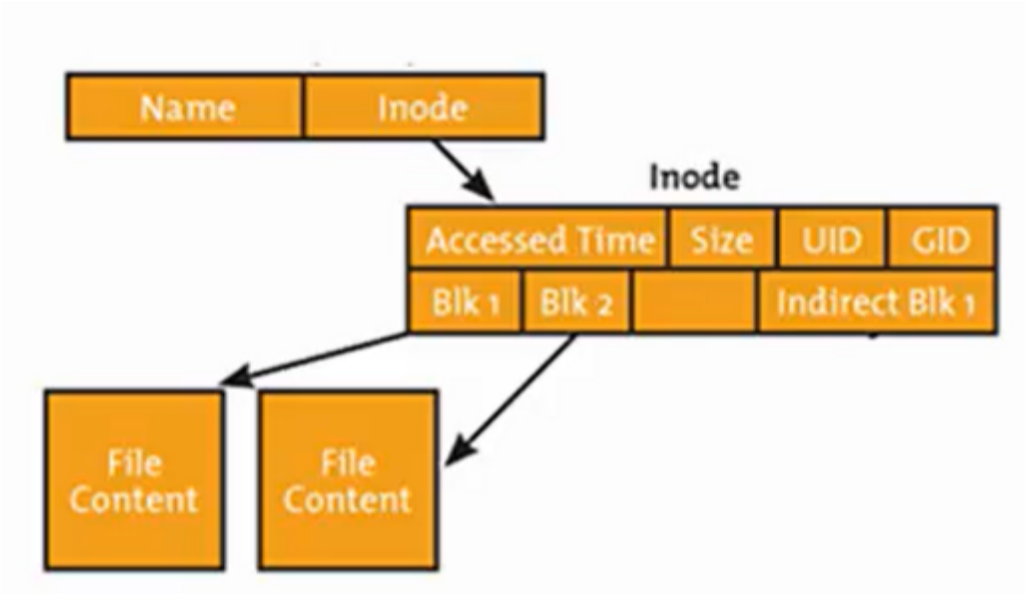
```
/dev/sda2 [device]     .....       ..     ....      ..    /
/dev/sda3                                                 /
home
/dev/sda1                                                 /
media/ec....

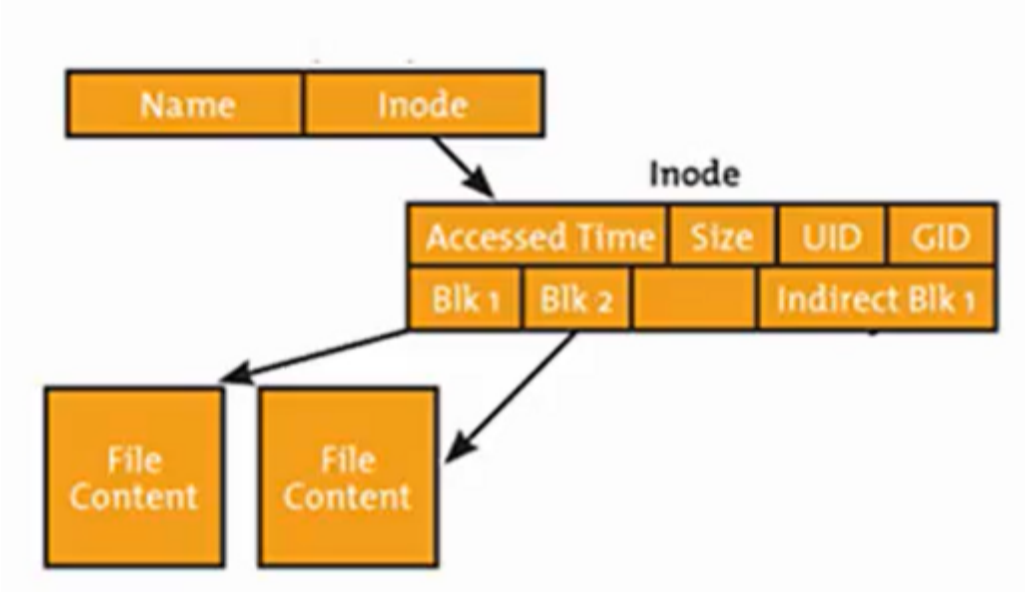
$df -i (Show FileSystem inode usage)


When is the HD full?
zero storage!
out of Inodes[try it]
Data content mla el HDD
```

File Operations

Copying a File





Renaming a File

inode is the same.

Just rename the name in the dentry !

Moving a File (Within Same FSs)

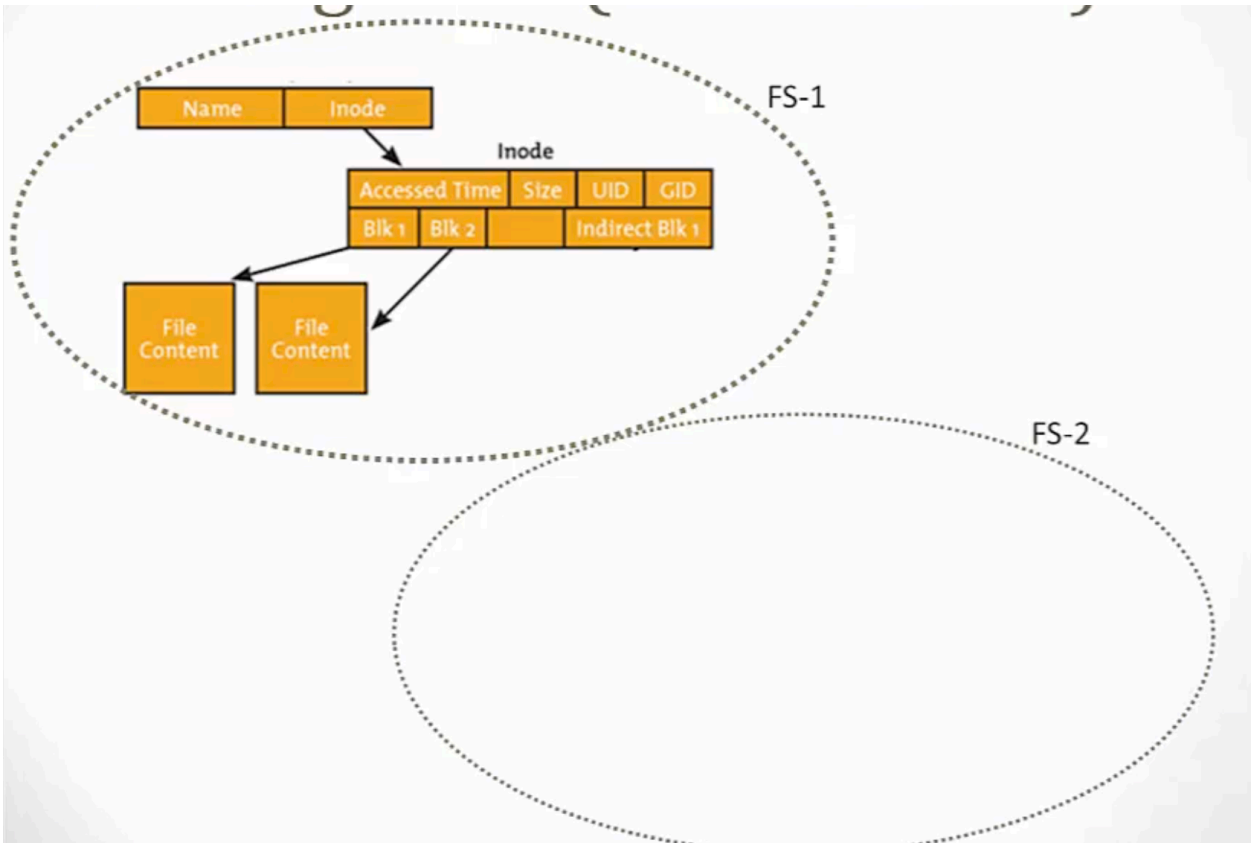
Just remove the old dentry

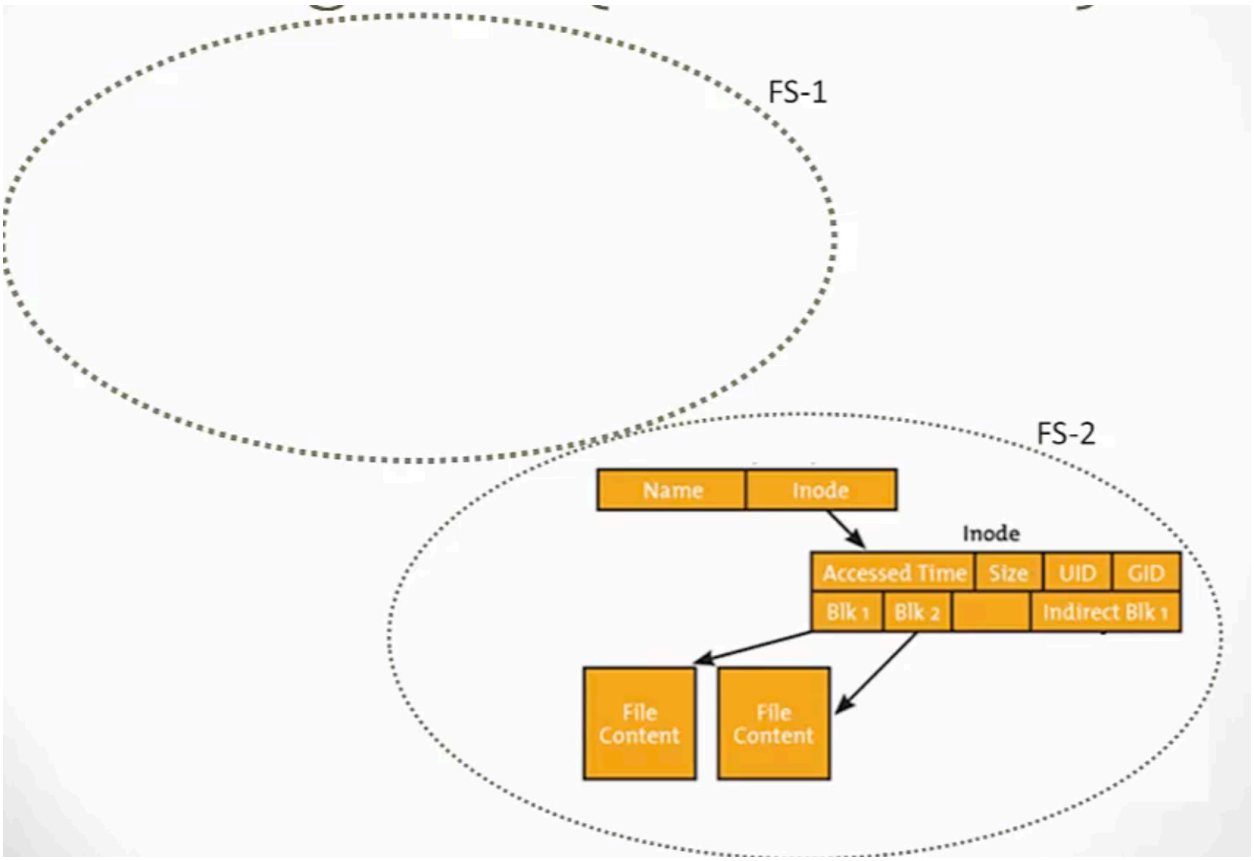
Create a new dentry

Moving a File (Between different FSs)

From HD to Flash

Move everything.


FS-1
Name
Inode
Inode
Accessed Time
Size
UID
GID
Blk 1
Blk 2
Indirect Blk 1
File Content
File Content
FS-2



Linking Files

```
/usr/share/results.log

lw ana dlw2ty 3ndy program hy2ra el log da whyb2a 3ayz y3ml 3
leh operations mo3yna
f 3ayz el program bta3y yb2a shayf el log da f h3ml eh
```

- Copy the file 3ndk
  - elmoshkla dlw2ty eno b2a duplicated, wmmkn t3dl f mkan tnsa eltany w hakza
- Shortcut leh zy mabn3ml fl Windows
  - Create a new dentry that points to the same inode. [Hard Link]

        - /home/ramy/a.log
    - Hard Links hya lly khlt enhom yfslo el name mn el inode.

Hard Links

The decision of not including the filename and path in the “inode” structure was to enable the use of hard links.

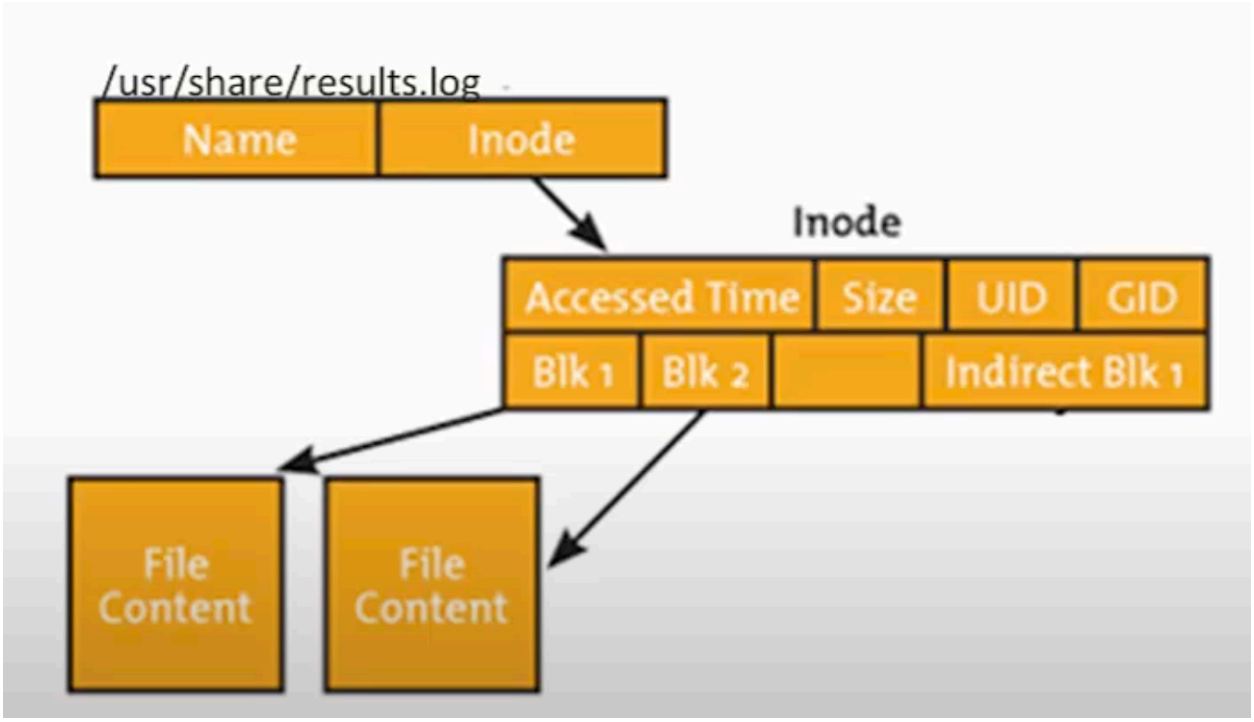
Hard links were introduced from the **early days of Unix**

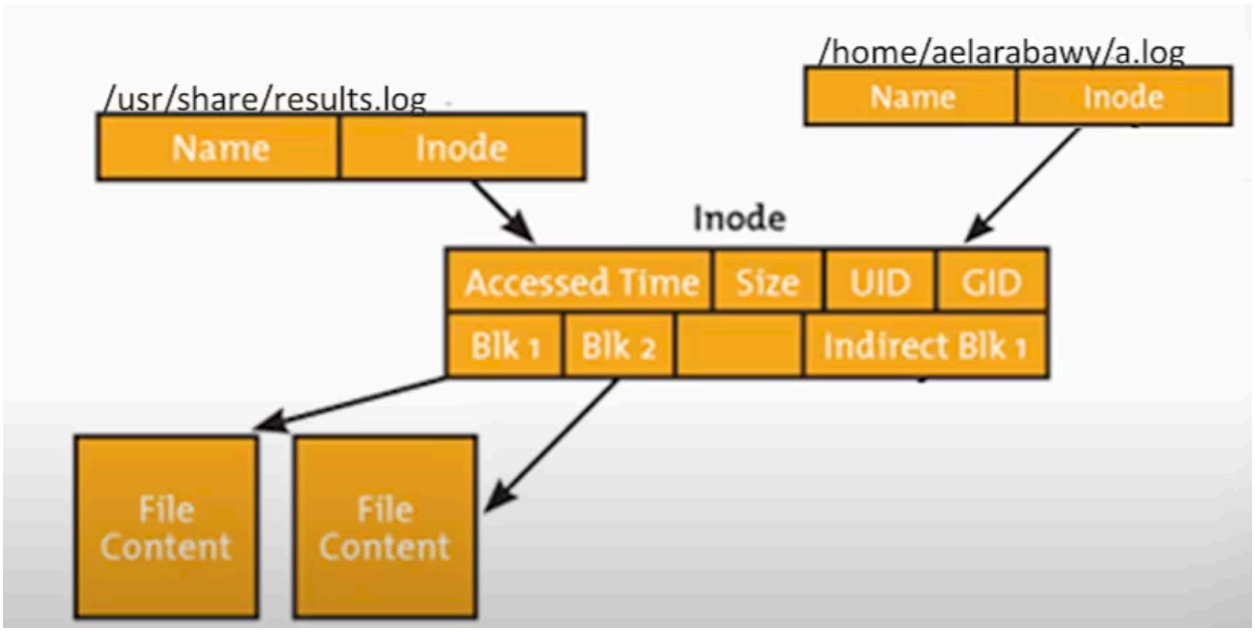
- A hard link
    - Not a new file
    - Same file content
    - Same **inode**

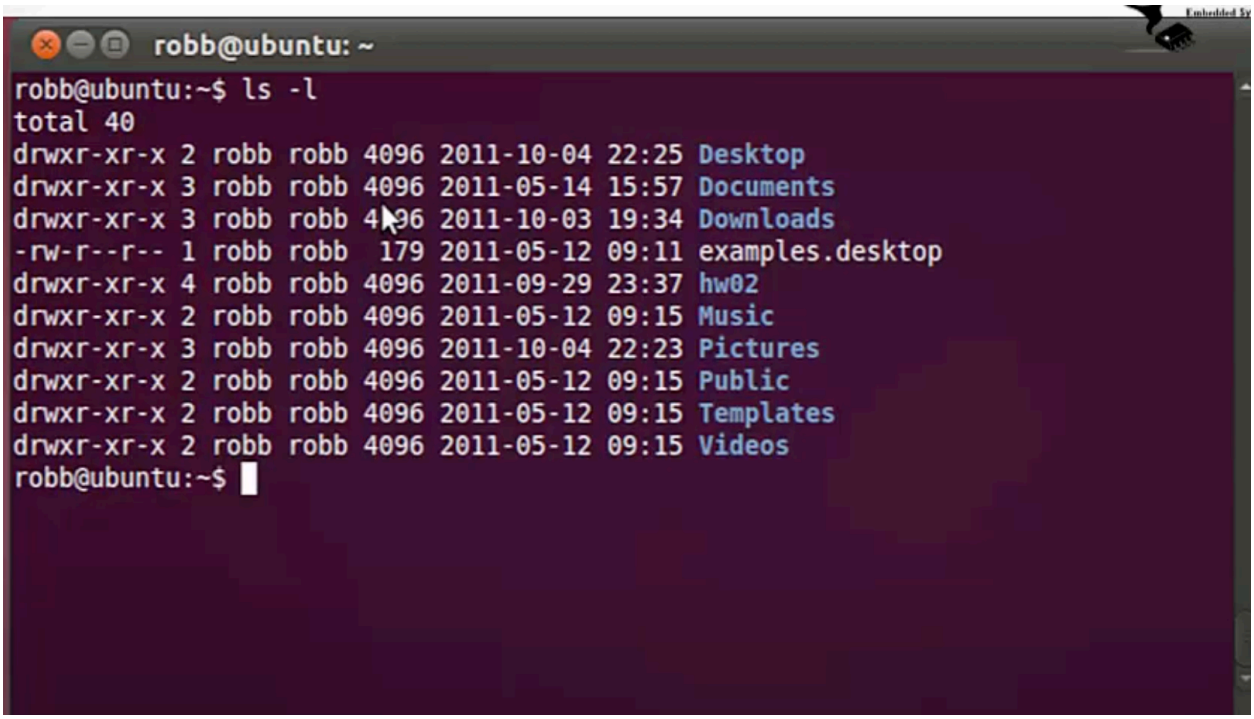
  Just an additional `“dentry”` with a different filename/path, but with the same `inode#`

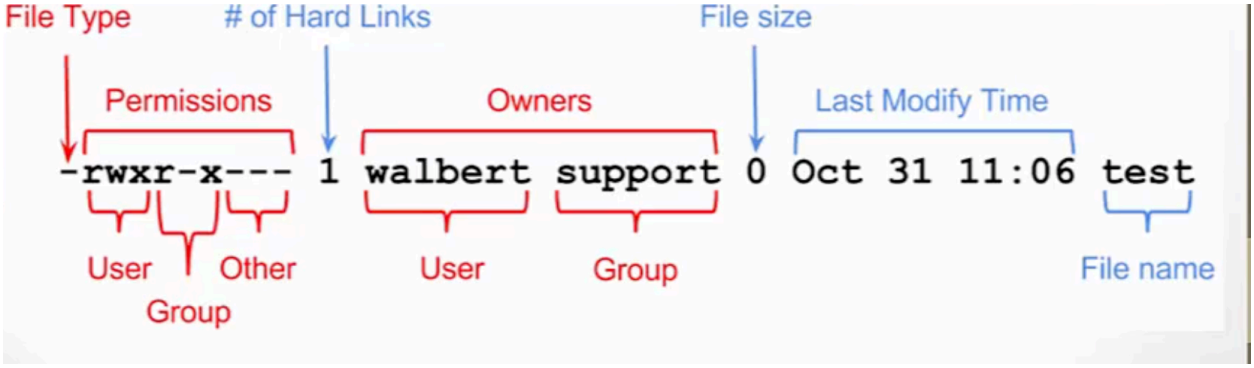
This is useful if we need to have the same file with two names, or in two locations.

```
$ls -l
# it prints the number of hard links
```


/usr/share/results.log
Name
Inode
Inode
Accessed Time
Size
UID
GID
Blk 1
Blk 2
Indirect Blk 1
File Content
File Content


/home/aelarabawy/a.log
Name
Inode
/usr/share/results.log
Name
Inode
Inode
Accessed Time
Size
UID
GID
Blk 1
Blk 2
Indirect Blk 1
File Content
File Content





However,

Hard links are **not very common** these days, they have some limitations,

- Only applicable for files, not used for directories.
  - After implementation it for directories, a **security hole was found**.
  - Can cause loops of links which result in system faults.
  - So it was disabled in latest releases.
- Does not work across filesystems.
  - We just link using the **inode#**

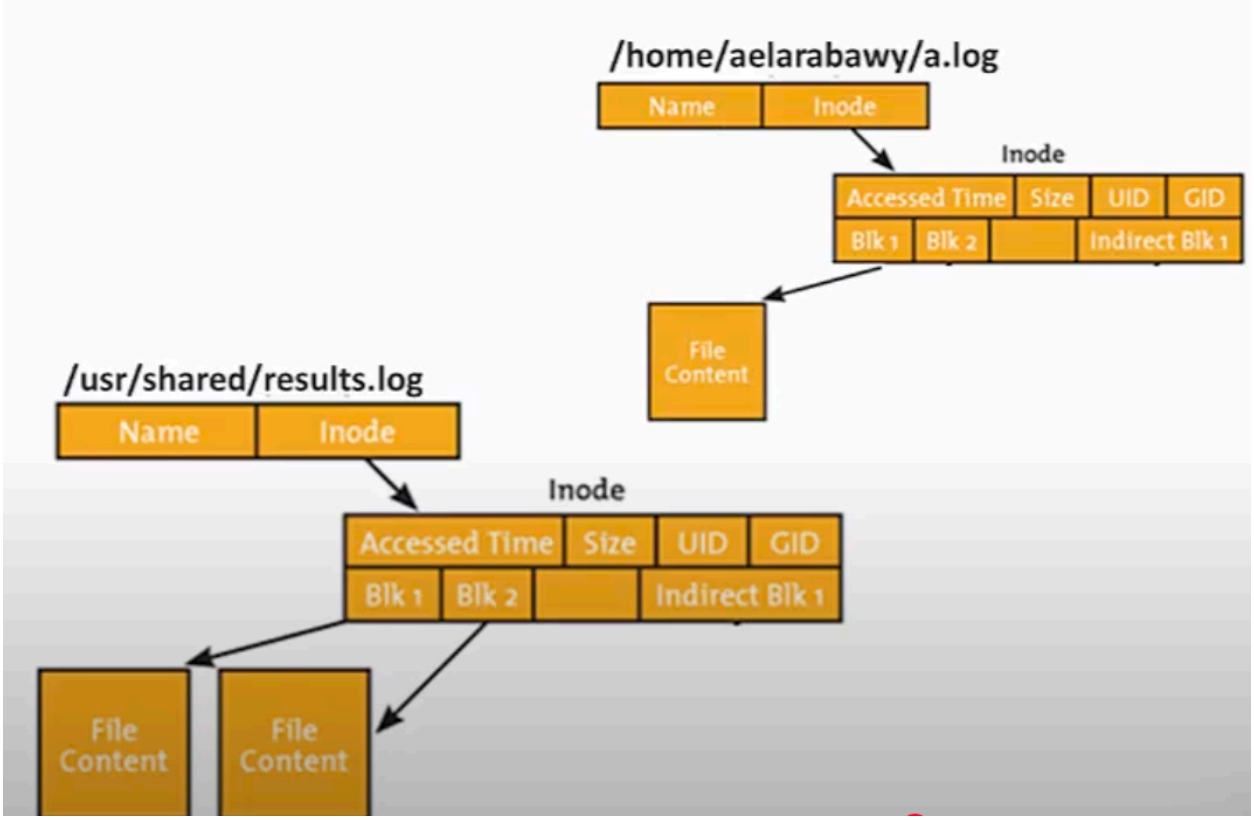
- But **inode#** is only **unique** within the same filesystem.
- Hence, we can not link to a file in a different filesystem.
- This is very limiting, specially Linux merges all the FS in a unified tree.

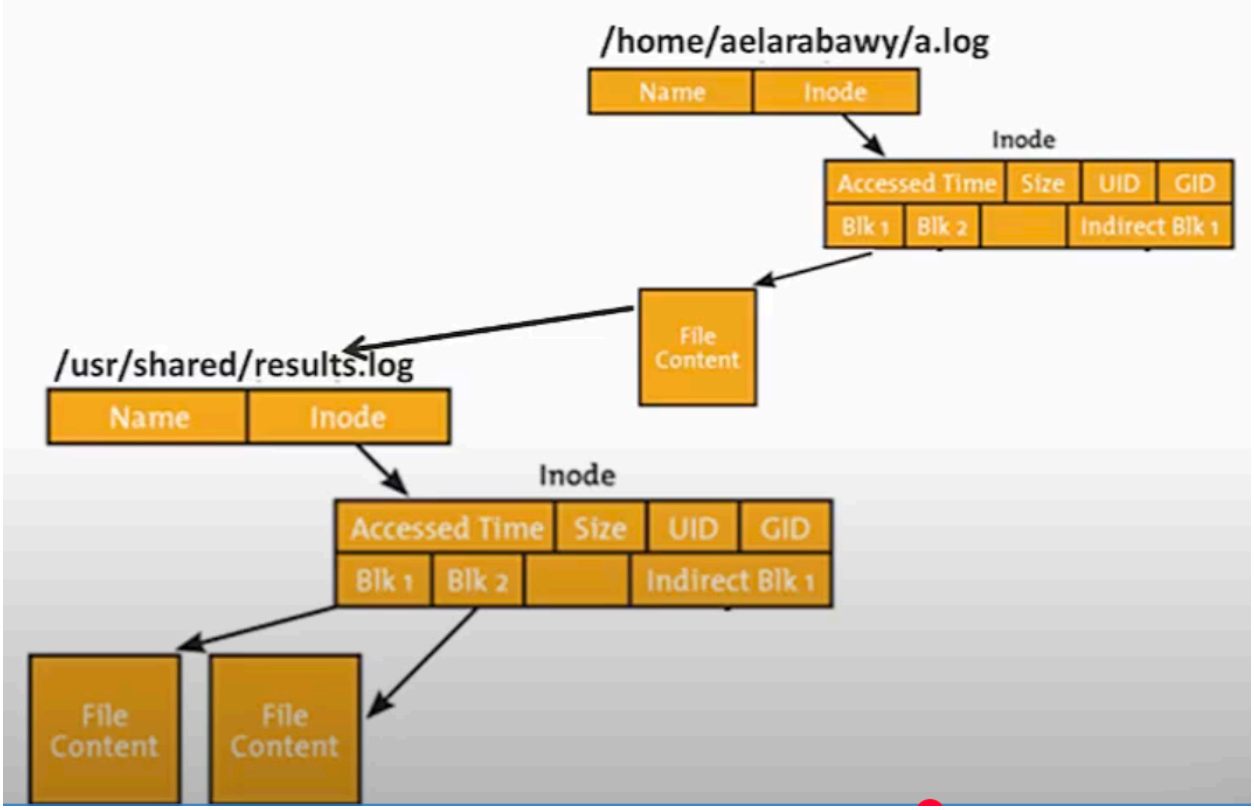
Symbolic Links

```
/usr/share/results.log

/home/ramy/a.log

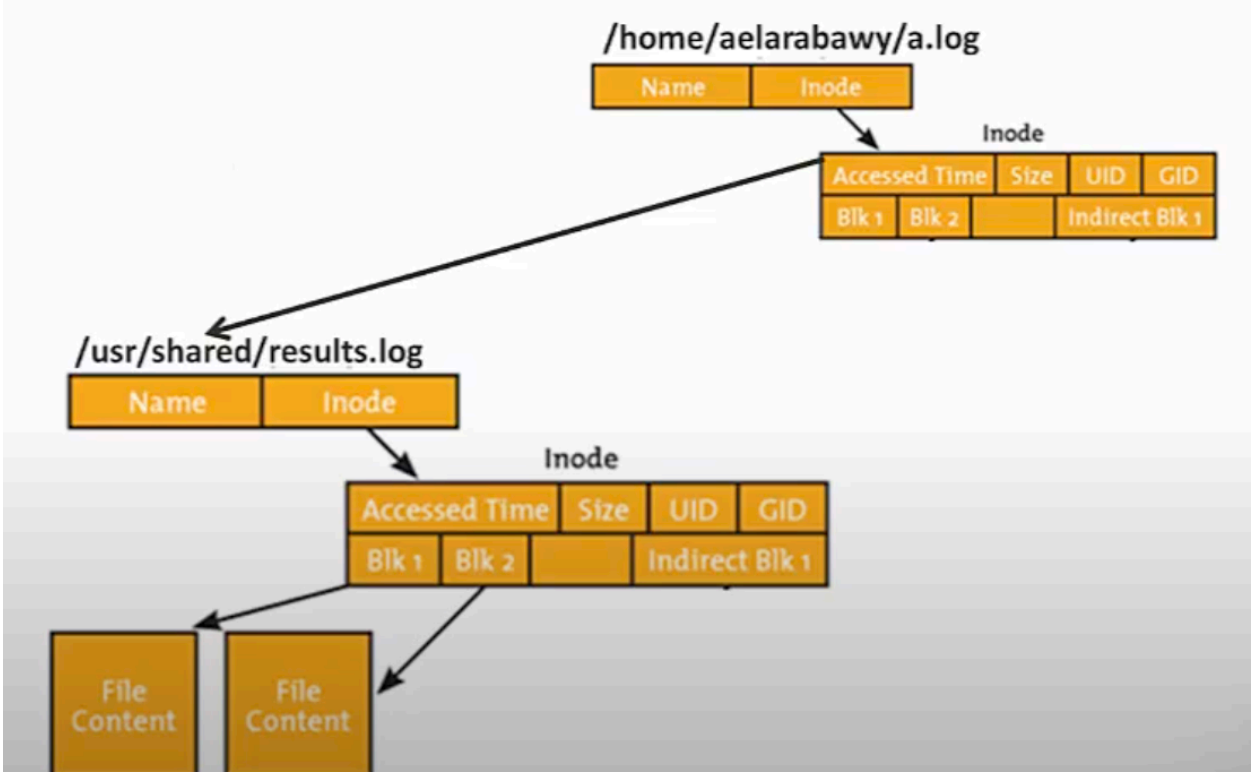
hy3ml eh b2a ?
hyro7 fl File block elgded hyktb el Name-inode el aslya w ykh
ly el mode Symbolic link
```





Fast Symbolic Links [Faster]

hy7ot field gded fl struct bta3 el inode y7ot feh el path 3la tol




```
andrew@D630:~/labs$ ls -l
total 8
lrwxrwxrwx 1 andrew andrew   14 Sep  7 11:21 doc -> /usr/share/doc
-rw-rw-r-- 1 andrew andrew    0 Sep  7 10:48 file1
-rwxrwxr-x 1 andrew andrew    0 Sep  7 10:48 file.sh
prw-rw-r-- 1 andrew andrew    0 Sep  7 11:05 pipe
drwxrwxr-x 3 andrew andrew 4096 Mar  1  2013 tools
drwxrwxr-x 3 andrew andrew 4096 Jan 25  2013 usp
andrew@D630:~/labs$
```

Summary

- A **Symbolic link** is introduced to **fix** the problems of **Hard links**.
- A **Symbolic link** is not just a dentry structure; it is a file with an **inode** structure.
- The **inode** structure
    - The type is set to '**l**' for a **symbolic link**

- Two types of Implementation:
  - **Slow** Symbolic Links:
    - The data block of the new file include the path of the file it is linking to
  - **Fast** Symbolic Links:
    - A field in the `inode` points to the path ad name of the file/directory it is pointing to
    - Faster, no need to read the data block.
    - Not possible if the path is too long to fit in the `inode` structure.
- Since a symbolic link it has its own inode, with an obvious indication that it is a link,
  - Some commands is able to treat it differently.
  - Avoid the security hole in hard links with linking directories.
- Symbolic links are like shortcuts in Windows.
- You can have a symbolic link to a file or a folder.

```
$ls -l
# [14] characters => usr/share/doc
```

Deleting a File

Can I distinguish who is the original file? If I have more than 1 Hard link ?

```
$ rm /usr/share/results.log
```

el kernel msh hyms7 content el file 8er lw kol el links khlst !

```
$ rm /usr/share/results.log
$ rm ~/a.log
```

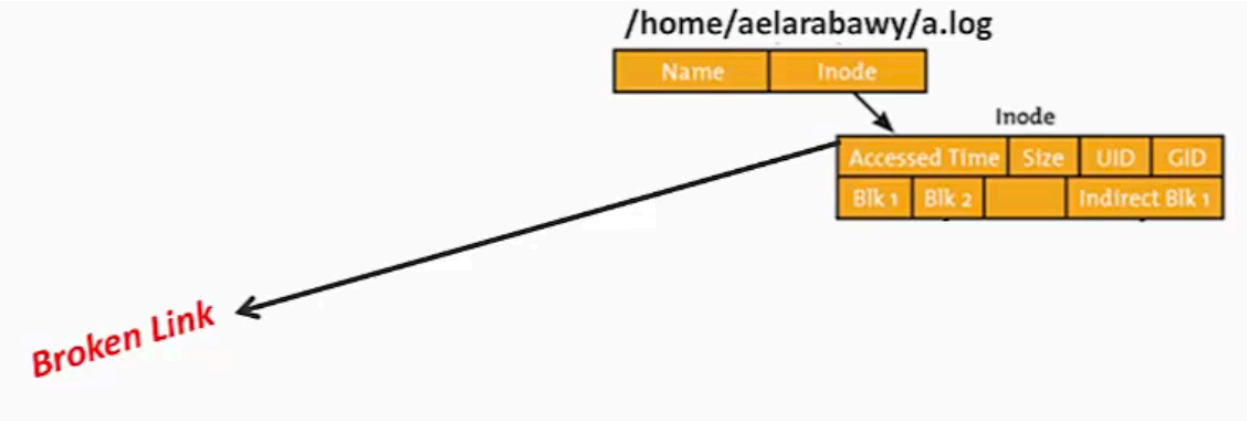
2a2dr amayz ben el file I asly wl link fl Symbolic Links ?

Yes !!

```
#lw ms7t el link

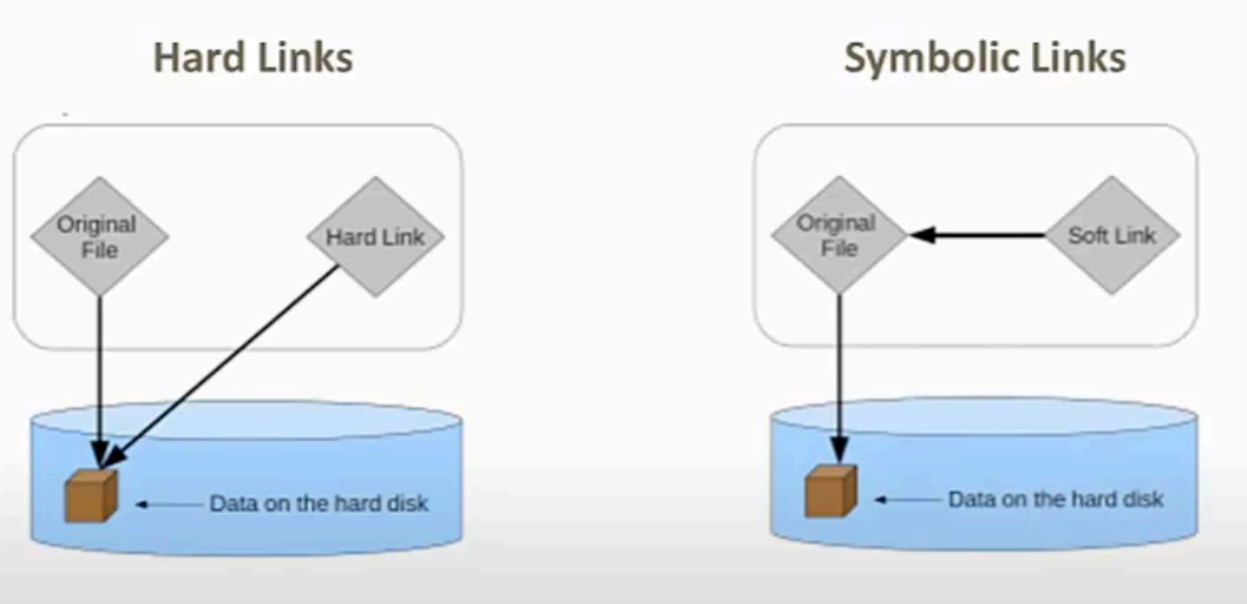
$rm ~/a.log

# lw ms7t el file msh el link?
broken/dangling link !! [by3mlk el link bl lon el a7mr]
tb lw rg3t el file aw 3mlt file bnfs l esm khalas 3ady hyzbot
El link shayf el file bs el file msh shayf el link
```



Which type of link is more space efficient ?

Hard Link Vs Symbolic Link (Abstracted View)



In Commands

```
# To create a Hard Link
$ ln <File to link to> <link name & location>
$ ln file.log ~/log-files/a.log

# To create a Symbolic Link
$ ln -s <File to link to> <link name & location>
$ ln -s ~/file.log ~/log-files/a.log

# Important notes
Always use absolute paths for the file to link to when creati
ng symbolic links...
Never use relative path format.
```

Question...

We agreed that Hard links are not allowed for directories...

Now doing a simple listing results in,,